

CleanNotes（清记）系统架构文档 v1.3.0

文档来源：基于 v1.3.0 源代码反向工程分析
分析日期：2025-07-10
分析方法：逐文件阅读 package.json / supabase.ts / hybrid.ts / storage.ts / auth.ts / useSync.ts / useTheme.ts / supabase-schema.sql / vite.config.ts 以及所有 stores / services / composables / types

1. 技术栈总览

1.1 前端

分类	技术	版本	用途
框架	Vue 3 (Composition API)	3.5.13	UI 框架，全部使用 <code><script setup></code> 语法
语言	TypeScript	5.7.2	静态类型检查
状态管理	Pinia	3.0.2	全局状态管理（8 个 Store）
UI 样式	TailwindCSS	4.1.0	原子化 CSS，通过 Vite 插件集成
富文本编辑器	Tiptap (基于 ProseMirror)	3.26.1	备忘录编辑、周报编辑
路由	Vue Router	4.5.0	Hash 模式（ <code>createWebHashHistory</code> ）
构建工具	Vite	6.2.0	开发/构建，含 <code>rollup-plugin-visualizer</code>
图表	Mermaid	11.16.0	Markdown 内嵌流程图/时序图渲染
脑图	markmap (markmap-lib + markmap-view)	0.18.12	Markdown → 思维导图
导出	html2canvas-pro	2.2.1	截图导出
XSS 防护	DOMPurify	3.4.9	HTML 内容清洗
Markdown 解析	marked	18.0.5	Markdown → HTML
农历	lunar-javascript	1.7.7	农历日历

1.2 桌面壳

技术	版本	用途
Tauri 2.0	@tauri-apps/api 2.3.0, @tauri-apps/cli 2.3.0	Windows/macOS/Linux 桌面壳

1.3 后端即服务 (BaaS)

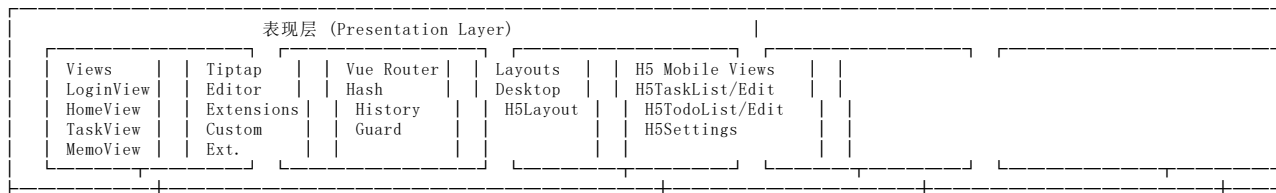
服务	说明
Supabase	PostgreSQL 16 + PostgREST REST API + RLS 行级安全策略
Supabase Storage	附件存储（cleannote_attachments bucket）

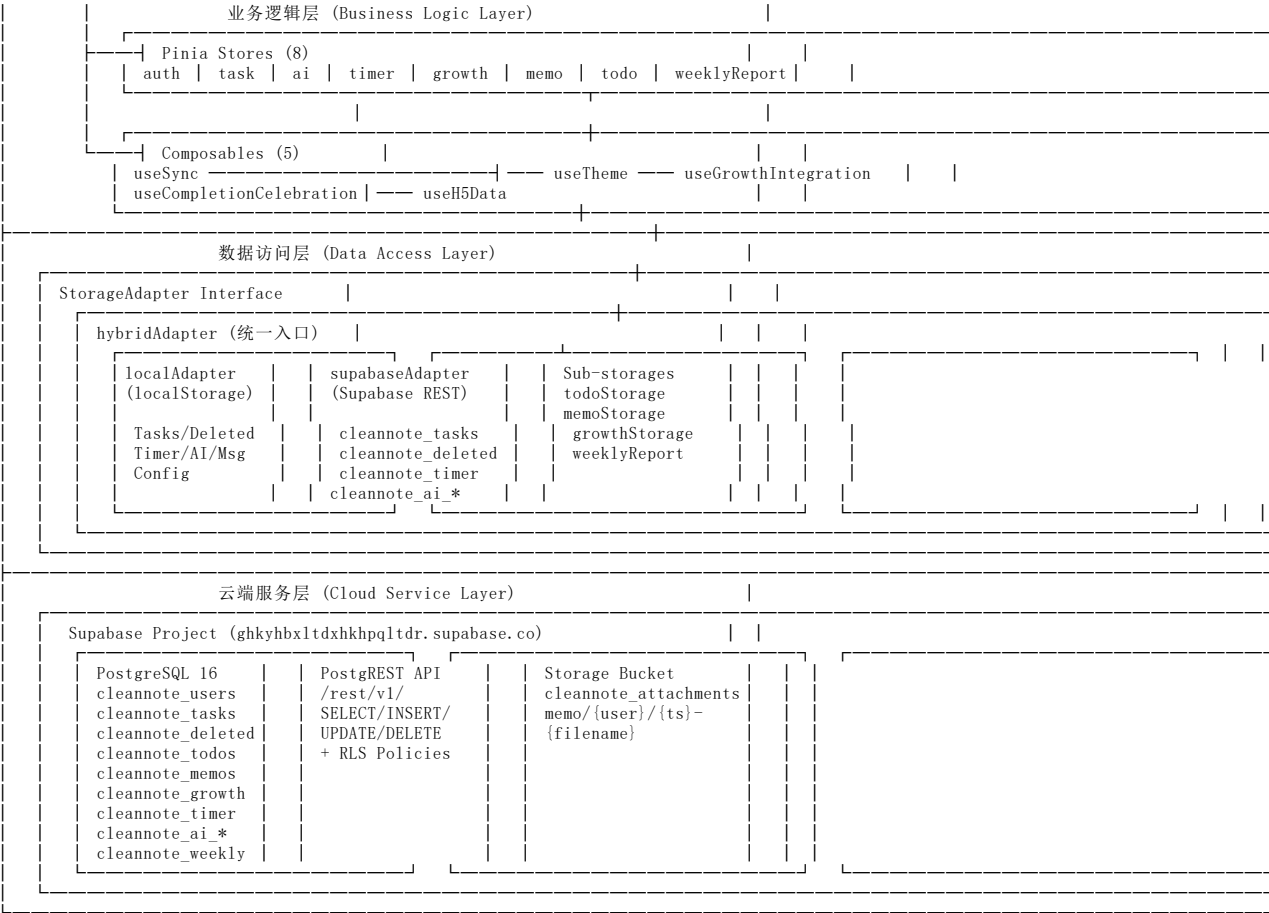
1.4 桌面打包

方案	说明
NSIS	Windows 安装包
IIS 静态托管	Web 端部署方案

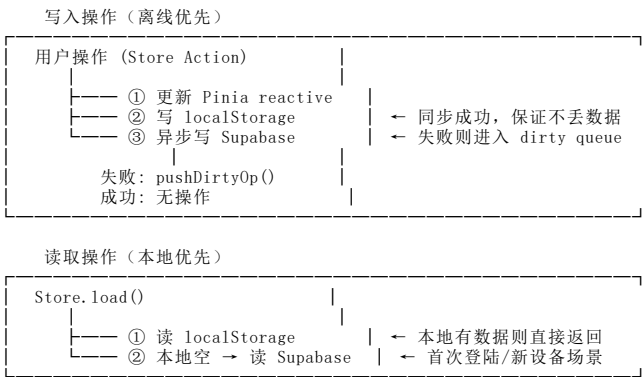
2. 系统架构图

2.1 四层架构



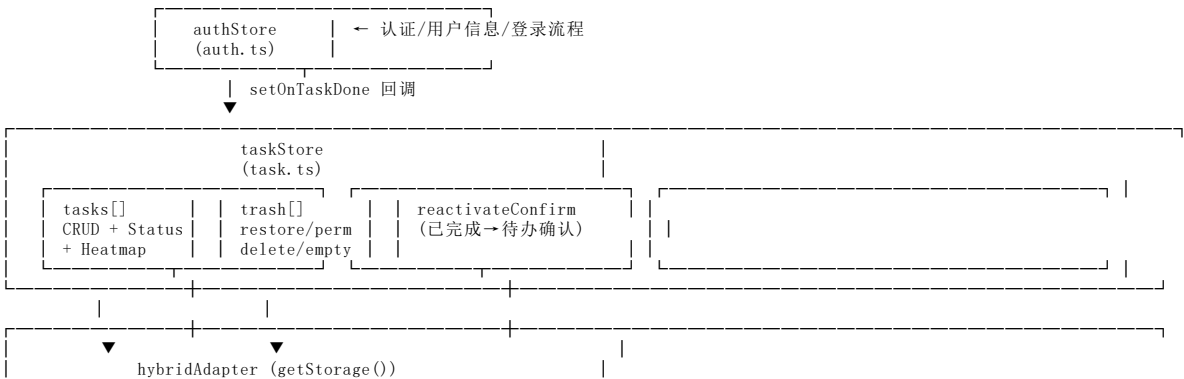


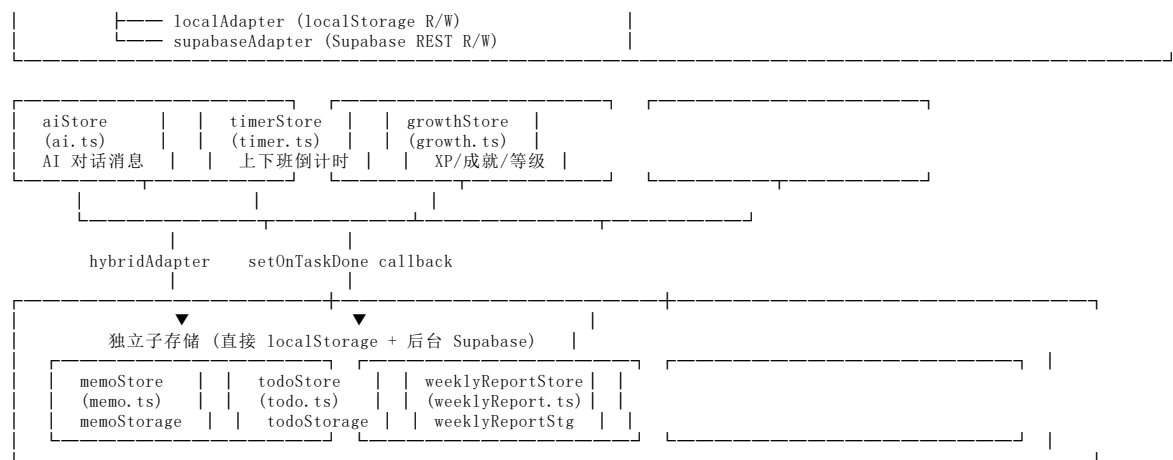
2.2 数据流方向



3. 核心模块依赖关系

3.1 8 个 Store 的依赖图

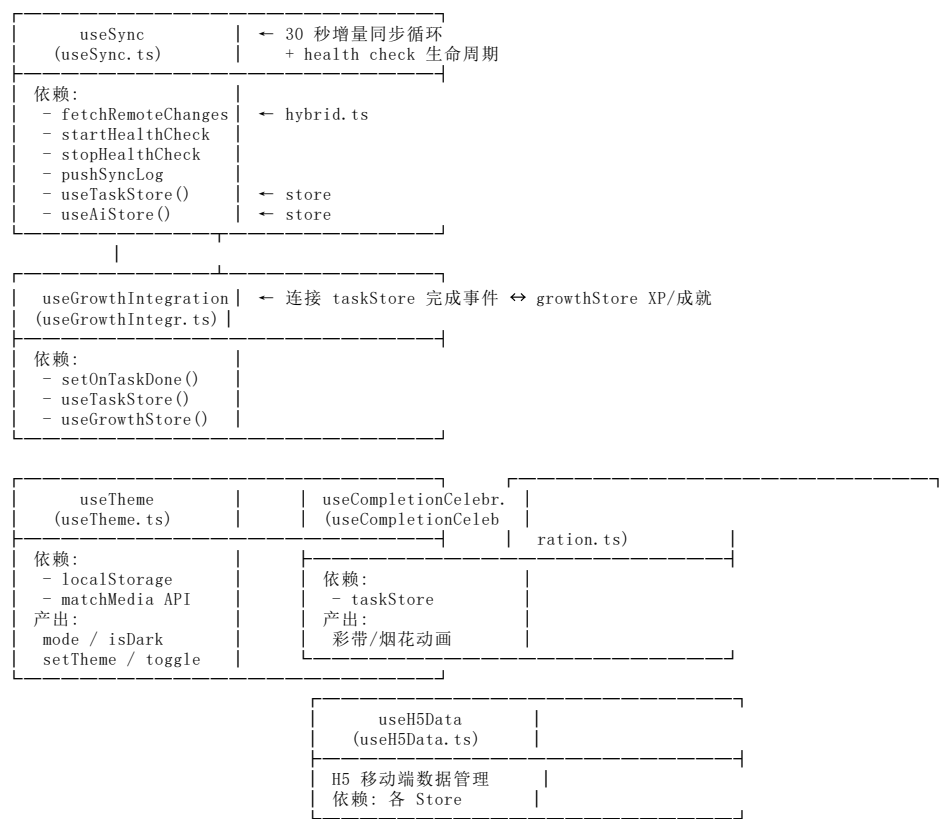




Store 间交互关系：

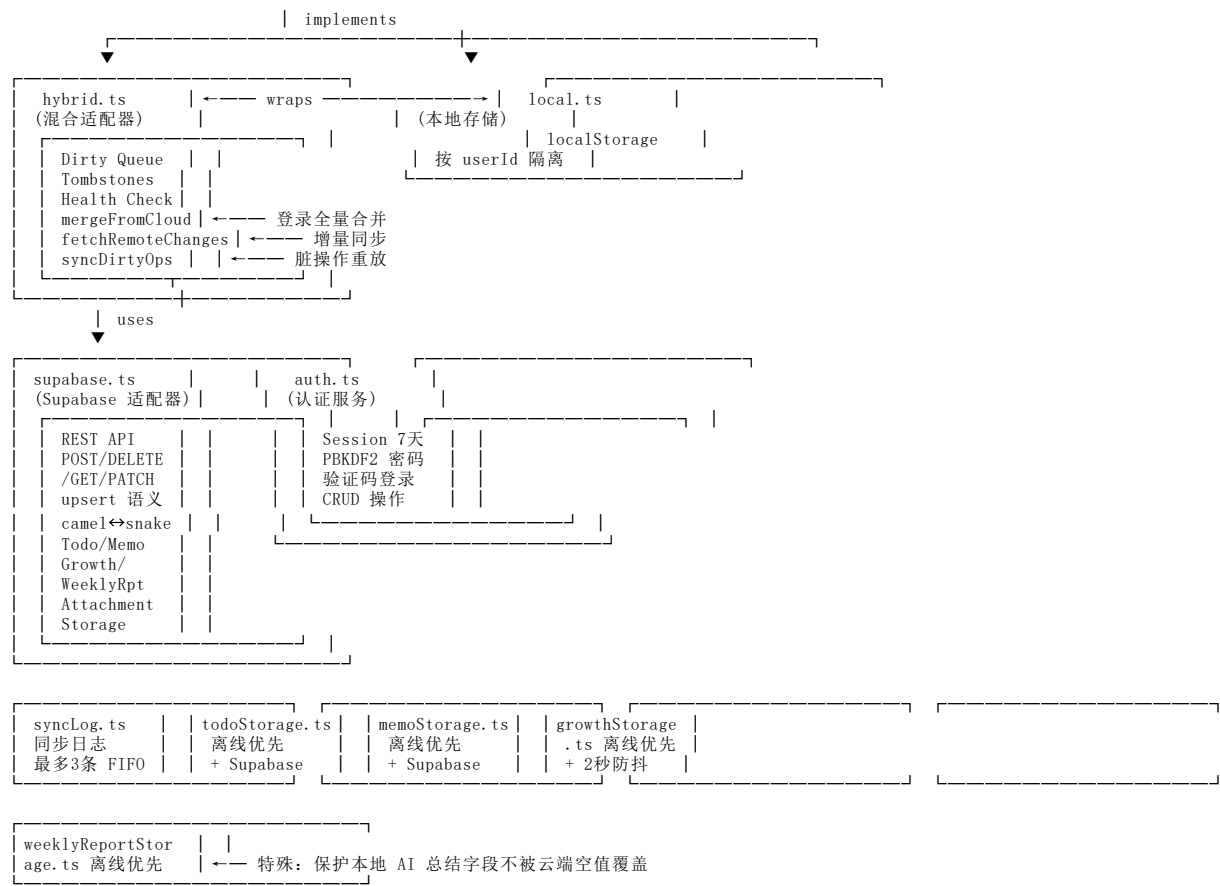
Store	依赖	被依赖
authStore	auth.ts (service)	Router Guard, 所有 Store (userId)
taskStore	getStorage() → hybridAdapter, hybrid.ts (tombstones)	growthStore (via callback)
growthStore	growthStorage.ts, taskStore (tasks via callback)	无
aiStore	getStorage() → hybridAdapter	无
timerStore	getStorage() → hybridAdapter	无
memoStore	memoStorage.ts (独立)	无
todoStore	todoStorage.ts (独立)	无
weeklyReportStore	weeklyReportStorage.ts (独立)	无

3.2 5 个 Composable 的依赖图



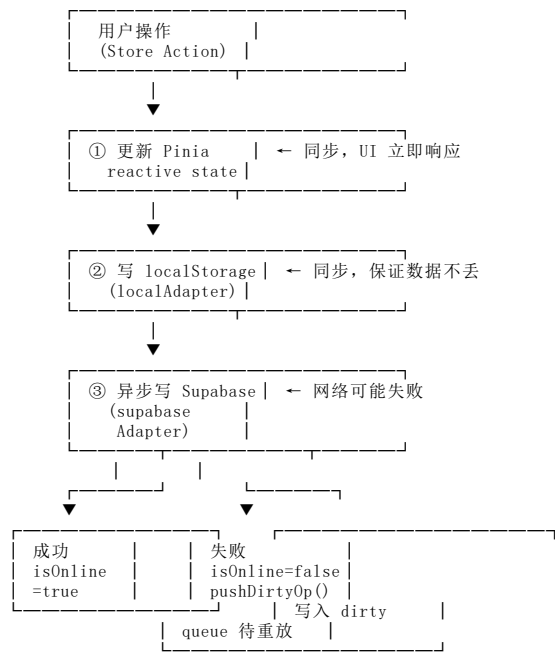
3.3 10 个 Service 的依赖图



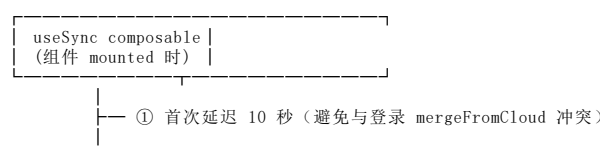


4. 数据流设计

4.1 离线优先策略（写操作）



4.2 增量同步机制（30 秒轮询）

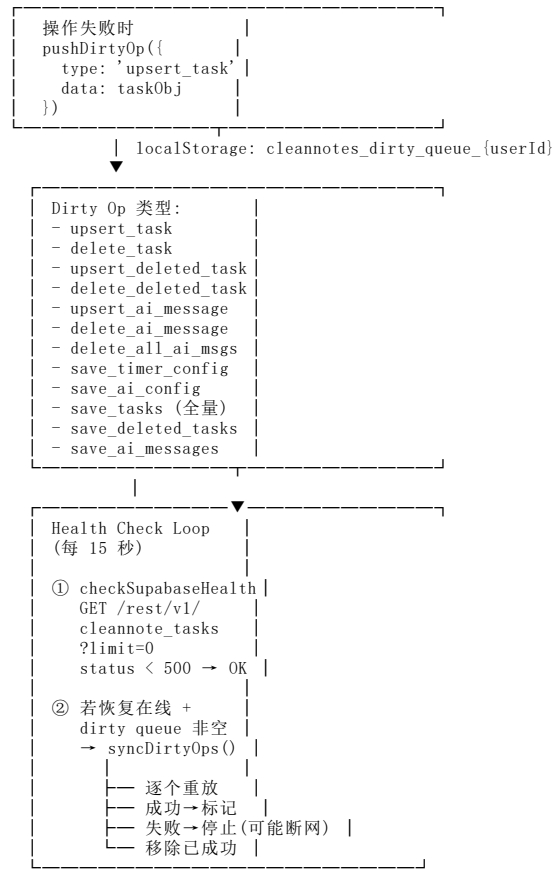




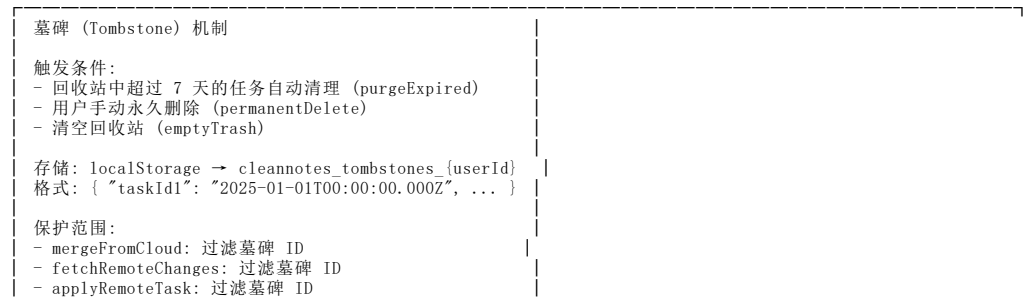
增量同步中的特殊处理：

- 脏队列保护：正在等待同步到云端的任务不会被标记为"远程已删除"（pendingSyncTaskIds）
- 孤儿清理：如果任务 ID 同时存在于 cleannote_tasks 和 cleannote_deleted_tasks，增量同步会 fire-and-forget 从 cleannote_tasks 中删除它
- 墓碑过滤：永久删除的任务 ID 永远不会被恢复

4.3 脏标记与重放机制



4.4 墓碑机制 (Tombstone)



```
- applyRemoteDeletedTask: 过滤墓碑 ID
生命周期:
- 最多保留 90 天 (TOMBSTONE_MAX_AGE_DAYS)
- 恢复任务时从墓碑中移除 (removeTombstones)
- 用户切换时自动清理过期墓碑 (cleanOldTombstones)
```

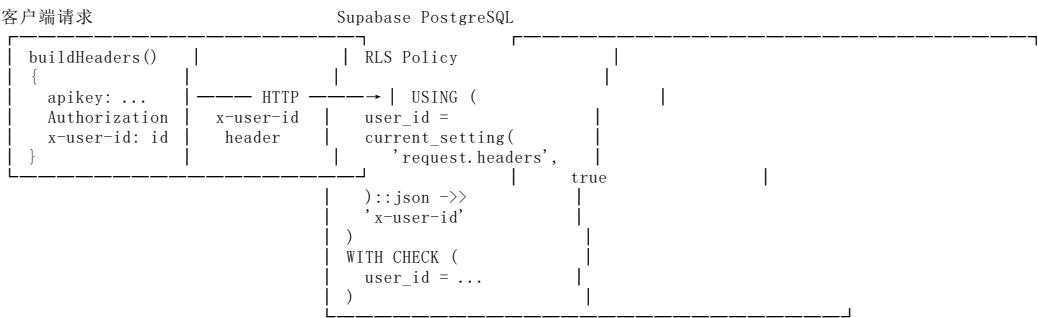
4.5 登录时全量合并流程 (mergeFromCloud)

```
用户登录成功
├── 1. 并行拉取 Supabase 全量数据
│   ├── cloudTasks / cloudDeleted / cloudMsgs
│   └── cloudTimer / cloudAiConfig
├── 2. 重新读取本地最新数据 (避免覆盖并行操作)
│   └── freshLocalTasks / freshLocalDeleted / etc.
├── 3. 合并任务 (Merge Tasks)
│   ├── 构建 deletedIdsSet (回收站中的 ID)
│   ├── 构建 tombstoneSet (墓碑 ID)
│   ├── 过滤: 移除已删除/墓碑任务
│   ├── 清理孤儿记录 (云端有但已被删除/墓碑标记)
│   └── mergeRecords():
│       ├── 仅云端有 → 拉取到本地
│       ├── 仅本地有 → 上传到云端
│       ├── 两端都有 → updatedAt 新者胜出
│       └── 时间戳相同 → 合并字段 (cloud base + local override)
├── 4. 合并回收站 (Merge Deleted Tasks) - 同上逻辑
├── 5. 合并 AI 消息 (Merge AI Messages) - 同上逻辑
├── 6. 合并 Timer/AI Config (云端优先覆盖)
│   ├── cloudTimer && !freshLocal → 采纳云端
│   ├── !cloudTimer && freshLocal → 上传本地
│   └── 两者都有 → 云端胜出
├── 7. 保存 lastSyncAt 时间戳
└── 8. 返回 hasChanges (bool) → Store.reload() 如果为 true
```

5. 安全架构

5.1 RLS 行级安全策略

Supabase PostgREST 自定义 Header 身份传递机制:



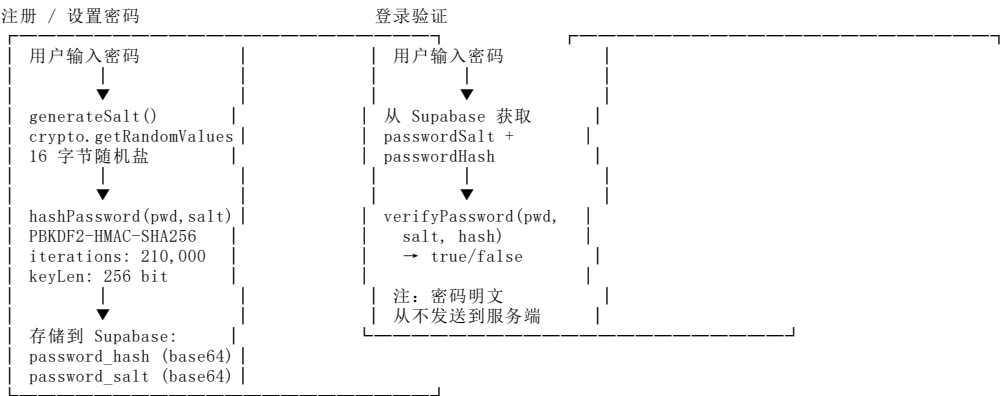
已启用 RLS 的表 (共 6 张):

表名	策略	说明
cleannote_users	FOR ALL USING (id = x-user-id)	用户只能访问自己的资料
cleannote_tasks	FOR ALL USING (user_id = x-user-id)	用户只能 CrUD 自己的任务
cleannote_deleted_tasks	FOR ALL USING (user_id = x-user-id)	用户只能 CrUD 自己的回收站
cleannote_timer_config	FOR ALL USING (user_id = x-user-id)	用户只能 CrUD 自己的配置
cleannote_ai_messages	FOR ALL USING (user_id = x-user-id)	用户只能 CrUD 自己的 AI 消息
cleannote_ai_config	FOR ALL USING (user_id = x-user-id)	用户只能 CrUD 自己的 AI 配置

安全说明 (来自 SQL schema 注释): 当前使用 anon key + x-user-id header 过滤, 安全性取决于客户端正确传递 user_id。后续可升级为 Supabase Auth + JWT。

5.2 PBKDF2-HMAC-SHA256 密码哈希

密码处理流程（基于 auth.ts 和 utils/crypto.ts 实现）：



关键安全设计： - 密码明文永不发送到服务端（auth.ts:109 注释："密码明文永不发送至服务端"） - 每用户独立随机盐（crypto.getRandomValues，16 字节） - PBKDF2 迭代次数 210,000（符合 OWASP 2025 推荐） - 密码强度要求：至少 8 位（auth.ts:149） - 遗留账号兼容：支持首次设置密码（password_hash/password_salt 为 NULL 时）

5.3 x-user-id Header 身份传递

```
// supabase.ts - buildHeaders()
function buildHeaders(): Record<string, string> {
  return {
    apikey: SUPABASE_KEY,
    Authorization: `Bearer ${SUPABASE_KEY}`,
    'Content-Type': 'application/json',
    ...(currentUserId ? { 'x-user-id': currentUserId } : {}),
  }
}

// auth.ts - registerUser 时手动设置（新用户尚无 currentUserId）
await request('cleannote_users', 'POST', {
  body: { id, phone, ... },
  userId: id, // 直接传递 userId 参数
})
```

5.4 会话管理

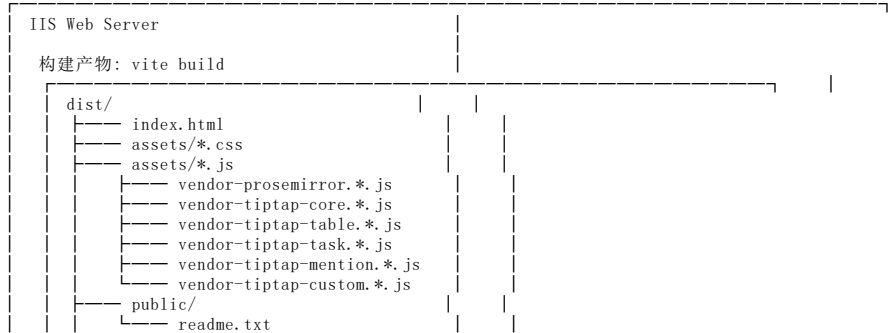
Session（存储在 localStorage）

interface Session {
 userId: string // 用户 ID
 phone: string // 手机号
 nickname: string // 昵称
 loginAt: number // Unix timestamp ms
 expiresAt: number // loginAt + 7 天
}

生命周期：
- 保存：登录/注册成功时 saveSession(user)
- 验证：应用启动时 getSession() 检查 expiresAt
- 过期：Date.now() > session.expiresAt → 清除 session
- 清除：退出登录 clearSession(), 修改密码后不失效
- 更新：修改昵称后同步更新 session 中的 nickname

6. 部署架构

6.1 IIS 静态托管方案



```
| | | stats.html (ANALYZE=true) | | |
|-----|
| base: './' (相对路径, 兼容子目录部署) |
```

6.2 NSIS 打包方案

```
NSIS Installer

来源: src-tauri/
Tauri 2.0 构建

目标平台:
- Windows x64
- macOS (Apple Silicon)
- Linux
```

6.3 H5 移动端部署

```
移动端架构 (自适应)

路由: /h5/* (H5Layout.vue wrapper)
|-----|
| /h5/tasks      H5TaskList.vue |
| /h5/tasks/new  H5TaskEdit.vue |
| /h5/tasks/:id  H5TaskEdit.vue |
| /h5/todos      H5TodoList.vue |
| /h5/todos/new  H5TodoEdit.vue |
| /h5/todos/:id  H5TodoEdit.vue |
| /h5/settings   H5Settings.vue |

设备检测: utils/device.ts
- isMobileDevice()
- isForcePC() / clearForcePC()

Router Guard 自动重定向:
- 移动端访问 PC 路由 → /h5/tasks
- 桌面端访问 H5 路由 → /home

数据共享: 同一套 Supabase backend
PC 和 H5 数据通过云端自动同步
```

6.4 环境变量与构建配置

构建时全局常量 (vite.config.ts):

```
define: {
  __APP_VERSION__: '1.3.0', // 从 package.json 读取
  __BUILD_TIME__: '2025-07-10 15:08', // 构建时生成
}
```

构建脚本:

命令	用途
vite build	标准生产构建 (含 vue-tsc 类型检查)
vite build --base /cleannotes-prod/	GitHub Pages 构建
bash scripts/copy-to-prod.sh	复制到生产环境
vite preview --host --port 4173	预览生产构建

代码分割策略 (manualChunks):

Chunk 名称	包含内容
vendor-prosemirror	ProseMirror 核心
vendor-tiptap-core	Tiptap 核心 + 基础扩展 + 图片缩放
vendor-tiptap-table	表格扩展 (table/row/header/cell)
vendor-tiptap-task	任务列表扩展 (task-list/task-item)
vendor-tiptap-mention	@提及扩展 (mention/suggestion)
vendor-tiptap-custom	自定义 Tiptap 扩展

Vite Dev Server: - 端口: 1420 (固定, strictPort: true) - HMR: 1421 (Tauri 开发模式) - 忽略: **/src-tauri/** (避免 Tauri 变更触发重载)

GitHub Pages 特殊构建 (build:gh-pages):

```
vite build --base /cleannotes-prod/
```


附录 A：项目文件结构

```
v1.3.0/
├── package.json          # 依赖与构建脚本
├── vite.config.ts        # Vite 构建配置
├── supabase-schema.sql   # 数据库 Schema + RLS
├── index.html            # 入口 HTML
├── docs/
│   └── 02-ARCH-系统架构文档.md # 本文档
├── scripts/
│   └── copy-to-prod.sh     # 生产部署脚本
├── src/
│   ├── main.ts            # Vue 应用入口 (Pinia + Router)
│   ├── App.vue            # 根组件
│   ├── style.css          # 全局样式 (TailwindCSS)
│   ├── env.d.ts           # 环境变量类型声明
│   ├── types/
│   │   └── index.ts       # 全部 TypeScript 类型定义
│   ├── router/
│   │   └── index.ts       # 路由配置 + 导航守卫
│   ├── stores/            # Pinia Stores (8 个)
│   │   ├── auth.ts       # 认证状态
│   │   ├── task.ts       # 任务管理
│   │   ├── ai.ts         # AI 对话
│   │   ├── timer.ts      # 倒计时
│   │   ├── growth.ts     # 成长系统
│   │   ├── memo.ts       # 备忘录
│   │   ├── todo.ts       # 待办事项
│   │   └── weeklyReport.ts # 周报
│   ├── services/         # 服务层 (10 个)
│   │   ├── storage.ts    # StorageAdapter 接口
│   │   ├── hybrid.ts     # 混合适配器 (核心同步逻辑)
│   │   ├── supabase.ts   # Supabase REST 客户端
│   │   ├── local.ts      # localStorage 适配器
│   │   ├── auth.ts       # 认证服务
│   │   ├── syncLog.ts    # 同步日志
│   │   ├── memoStorage.ts # 备忘录存储
│   │   ├── todoStorage.ts # 待办存储
│   │   ├── growthStorage.ts # 成长系统存储
│   │   └── weeklyReportStorage.ts # 周报存储
│   ├── composables/      # 组合式函数 (5 个)
│   │   ├── useSync.ts    # 同步机制
│   │   ├── useTheme.ts   # 主题管理
│   │   ├── useGrowthIntegration.ts # XP/成就连接
│   │   ├── useCompletionCelebration.ts # 完成动画
│   │   └── useH5Data.ts  # H5 数据
│   ├── utils/
│   │   ├── time.ts       # 时间工具 (normalizeTimestamp, toUTCISO)
│   │   ├── crypto.ts     # 加密工具 (PBKDF2 hash/salt)
│   │   └── device.ts     # 设备检测
│   ├── views/            # 页面组件
│   │   ├── LoginView.vue
│   │   ├── HomeView.vue
│   │   ├── TasksView.vue
│   │   ├── TodoView.vue
│   │   ├── MemoView.vue
│   │   ├── AiView.vue
│   │   ├── SettingsView.vue
│   │   ├── SpiritView.vue
│   │   ├── ReportsView.vue
│   │   └── DiagView.vue
│   ├── h5/               # H5 移动端页面
│   ├── components/       # 通用组件
│   ├── extensions/       # Tiptap 自定义扩展
│   ├── layouts/          # 布局组件
│   │   ├── DesktopLayout.vue
│   │   └── H5Layout.vue
│   └── assets/           # 静态资源
```

附录 B：Supabase 数据库 Schema 摘要

表名	主要字段	主键	索引
cleannote_users	id(UUID), phone(UNIQUE), nickname, password_hash, password_salt, created_at, last_login_at	id	-
cleannote_tasks	id, user_id(FK), title, description, status(ENUM), priority(ENUM), due_date, start_date, start_time, tags(JSONB), created_at, updated_at, completed_at, in_progress_at	id	user_id
cleannote_deleted_tasks	同 tasks + deleted_at	id	user_id
cleannote_timer_config	id(=1), user_id(UNIQUE FK), work_start, work_end, work_days(JSONB)	id	user_id
cleannote_ai_messages	id, user_id(FK), role(ENUM), content, timestamp	id	user_id
cleannote_ai_config	id(=1), user_id(UNIQUE FK), api_url, api_key, model	id	user_id

表名	主要字段	主键	索引
cleannote_todos	id, user_id(FK), title, description, estimated_start, estimated_end, linked_task_id, importance, created_at, updated_at	id	-
cleannote_memos	id, user_id(FK), title, content, tags(JSONB), pinned, icon, sort_order, created_at, updated_at	id	-
cleannote_growth	user_id(UNIQUE FK), state_json, xp_events_json, achievements_json, updated_at	user_id	-
cleannote_weekly_reports	id, user_id(FK), week_start, week_end, content, summary(JSONB), ai_summary, ai_summary_status, created_at, updated_at	id	-

外键级联：所有 user_id 外键均设置 ON DELETE CASCADE。

文档版本: v1.3.0
最后更新: 2025-07-10
维护者: CleanNotes 开发团队